Phase 3 – Decoupling the Application Components

Objective

The objective of Phase 3 was to separate the application and database components into independent infrastructure resources.

The Student Records application continues to run on an EC2 instance, while the database is migrated to Amazon RDS running inside private subnets.

This phase included:

- creating private subnets in multiple Availability Zones,
- creating an Amazon RDS MySQL database,
- configuring a DB Subnet Group,
- configuring database security groups,
- migrating the application database from EC2 to RDS,
- updating the application configuration,
- testing full CRUD functionality using Amazon RDS.

Created AWS Resources

Resource Type	Name
VPC	phase3-vpc
Public Subnet	10.0.1.0/24
Private Subnet AZ-1	10.0.2.0/24
Private Subnet AZ-2	10.0.3.0/24
Internet Gateway	phase3-igw
EC2 Instance	phase3-webserver
Amazon RDS MySQL	database-1
DB Subnet Group	RDS Subnet Group
Cloud9 Environment	phase3-cloud9

Architecture Changes

The main goal of this phase was to decouple the database from the application server.

In Phase 2, both the Node.js application and MySQL database were running on the same EC2 instance.

In Phase 3:

- the Node.js Student Records application remains on the EC2 instance,
- the MySQL database was migrated to Amazon RDS,
- the RDS instance was deployed inside a private subnet,
- a second private subnet was created to satisfy Amazon RDS multi-subnet requirements,
- communication between EC2 and RDS is allowed only through security group rules.

This architecture improves security and better reflects real-world cloud deployments.

Network Configuration

The existing VPC was reused with CIDR block:

10.0.0.0/16

The network contains:

Subnet	CIDR
Public Subnet	10.0.1.0/24
Private Subnet AZ-1	10.0.2.0/24
Private Subnet AZ-2	10.0.3.0/24

The EC2 application server remains inside the public subnet.

The Amazon RDS database was deployed inside the private subnet and is not publicly accessible from the internet.

Amazon RDS Configuration

A managed Amazon RDS MySQL database was created.

Configuration:

Setting	Value
Engine	MySQL
Database Name	STUDENTS

Setting	Value
DB Identifier	database-1
Instance Class	db.t4g.micro
Accessibility	Private
Deployment	Single-AZ
Storage	Default AWS Storage

The database was successfully created and became available for connections from the EC2 instance.

Database Initialization

Using AWS Cloud9 and MySQL client tools, a connection to Amazon RDS was established.

The following actions were completed:

- connected to the RDS endpoint,
- created the STUDENTS database,
- verified database creation,
- created the required tables,
- inserted initial student records,
- verified data using SQL queries.

Example verification commands:

```
SHOW DATABASES;

USE STUDENTS;

SHOW TABLES;

SELECT * FROM students;
```

Application Configuration

The Student Records application was updated to use the Amazon RDS endpoint instead of the local MySQL server.

The following environment variables were configured:

```
APP_DB_HOST=<RDS_ENDPOINT>
```

APP_DB_USER=admin

APP_DB_PASSWORD=<PASSWORD>

APP_DB_NAME=STUDENTS

APP_PORT=80

The Node.js application was then started and successfully connected to the RDS database.

Connection verification messages confirmed successful communication between the application server and Amazon RDS.

Security Configuration

Security groups were configured to allow:

Type	Port
HTTP	80
SSH	22
MySQL	3306

The MySQL port was restricted to communication between the EC2 instance and the RDS database.

The database remains inaccessible directly from the public internet.

Application Testing

After updating the application configuration, the Student Records application was successfully accessed through the EC2 public IPv4 address.

The following operations were tested successfully:

- viewing student records,
- adding student records,
- updating student records,
- deleting student records,
- verifying database changes inside Amazon RDS.

All CRUD operations worked correctly while using Amazon RDS as the backend database.

Infrastructure Diagram

An updated infrastructure diagram was created to reflect the new architecture.

Key changes compared to Phase 2:

- MySQL database removed from EC2,
- Amazon RDS added inside a private subnet,
- second private subnet added for DB Subnet Group,
- secure communication established between EC2 and RDS.

Screenshots

All screenshots related to Phase 3 are available inside:

docs/phase3/screenshots/

Final Result

At the end of Phase 3, the Student Records application was successfully decoupled into separate application and database components.

The Node.js application continues running on an EC2 instance in a public subnet, while the MySQL database runs securely on Amazon RDS inside private subnets.

This architecture provides improved security, better maintainability, and serves as the foundation for implementing high availability and scalability in future phases.